



IMPROVING FACE POSE ESTIMATION USING LONG-TERM TEMPORAL AVERAGING FOR STOCHASTIC OPTIMIZATION

Nikolaos Passalis and Anastasios Tefas
passalis@csd.auth.gr, tefas@aiia.csd.auth.gr

Stochastic Optimization

Proposed Method

Experimental Evaluation

Conclusion

STOCHASTIC OPTIMIZATION

- **Optimization** is at the heart of most Machine Learning (ML) techniques
 - The quality of a model is evaluated using a **loss function** $\mathcal{L}$
 - The parameters of a model are **learned** by **optimizing** $\mathcal{L}$
- For some methods, **closed-form** solutions do exist
 - Ridge Regression, Support Vector Machines (SVM), Principal Component Analysis (PCA), etc.
 - The global optimum is guaranteed to be found!

- Unfortunately, for most non-linear models **closed-form solutions are not available**
 - This includes most current state-of-the-art techniques!
 - **Deep Neural Networks**, Word Embedding Models, Neighbor Embedding Techniques (t-SNE), ...
- Even when they exist, calculating the closed-form solution might be **computationally prohibitive!**
 - In the **Big Data**-era it is difficult to calculate exact solutions even for simple models!
 - Can we fit an SVM on the Imagenet dataset?

- Usually, **Gradient Descent** is used to optimize the loss function $\mathcal{L}$:

$$\theta_{t+1} = \theta_t - \eta \frac{\partial \mathcal{L}}{\partial \theta_t} \quad (1)$$

where θ_t are the parameters of the model and η the used learning rate

- A local minimum is obtained if $\mathcal{L}$ is not convex
- The **whole training set** is used to accumulate the gradients before updating the parameters

- **Stochastic Gradient Descent (SGD)** is usually preferred due to **faster convergence!**
 - The parameters are updated using gradients only from **one (or a few) sample(s)** (instead of the whole training set)
 - **Mini-batches** are usually used to exploit the parallelism that modern CPU/GPU architectures provide
- The popularity of SGD led to several variants:
 - **SGD with momentum** to **accelerate** the convergence
 - **Averaged SGD** to reduce the **noisy nature** of the updates
 - **AdaGrad, RMSProp, Adam** to provide adaptive updates **per parameter**

- **Assisting drone-based cinematography!**
 - Flying a drone in a professional filming setting requires at least two operators
 - Use machine learning to **reduce the load of the human operators** and **increase the quality of the captured shots**
 - Train **Deep Convolutional Neural Networks (CNNs)** for **facial pose estimation**

- While training deep CNNs, we observed the following:
 - the **convergence was not smooth**, even when using state-of-the-art optimization techniques
 - the **noisy nature of the data** further contributes to this (the face image is cropped using a detector before estimating the pose)
 - the **order of the updates matters**
 - it is not easy to select when to stop (balance between **under-fitting** and **over-fitting**)

PROPOSED METHOD

- To remedy the noisy stochastic updates of gradient descent, **parameter averaging** techniques have been proposed
- **Parameter Averaging** (also known as Averaged SGD) was among the first naive attempts

$$\theta_{\text{final}} = \frac{1}{N} \sum_{t=1}^N \theta_t$$

- Slows down the convergence and **bias** the solution towards the initial estimation

- **Exponential Parameter Averaging** was proposed in the Adam's paper (but not evaluated)

$$\theta_{t+1} = (1 - \alpha)\theta_t + \alpha\theta_{t+1}$$

- The exponential moving average gives higher weight to the recent values of the parameters
- In the **Reinforcement Learning** community, similar parameter averaging has been also used **to avoid “forgetting” older experiences.**

- Instead of directly updating the network using the parameters obtained through SGD, we do the following:
 1. **Keep a “stable” versions of the parameters**
 2. **Proceeding using SGD for a number of iterations**
 3. After them, **anneal towards a stabler state** using the “stable” parameters
 4. **Save the updated “stable” parameters** and repeat
- The proposed technique:
 - **increases the convergence speed**
 - **better explores the solution space** leading to better solutions
 - **maintains the stability that the averaging process offers**

LONG-TERM TEMPORAL AVERAGING ALGORITHM

Input: A training set of data $\mathcal{X}$, the initial parameters of the network θ_0 , and an optimization method $f(\cdot)$ along with its hyper-parameters η

Parameters: the target update rate α_∞ , the update rate decay m , the exploration window N_S , and the number of iterations N

Output: The optimized parameters θ

```
1: procedure LONG-TERM TEMPORAL AVERAGING ALGORITHM
2:    $\theta_{\text{past}} \leftarrow \theta_0$ 
3:   for  $t \leftarrow 1; t \leq N; t++$  do
4:     Sample a batch  $x$  from  $\mathcal{X}$ 
5:      $\theta_t \leftarrow f(\theta_{t-1}, x, \eta)$ 
6:     if  $\text{mod}(t, N_S) = 0$  then
7:        $\alpha \leftarrow \alpha_\infty + (1 - \alpha_\infty) \exp(-m(t/N_S))$ 
8:        $\theta_{\text{past}} \leftarrow \alpha\theta_t + (1 - \alpha)\theta_{\text{past}}$ 
9:      $\theta_t \leftarrow \theta_{\text{past}}$ 
10:  return  $\theta_{\text{past}}$ 
```

Note that the update rate is larger during the initial iterations and slowly reduces to α_∞

LONG-TERM TEMPORAL AVERAGING ALGORITHM ANALYSIS

- Let's analyze the Long-Term Temporal Averaging updates!
- Consider the first N_S iterations:

$$\theta_1 = \theta_0 - \eta \frac{\partial \mathcal{L}}{\partial \theta_0}$$

$$\theta_2 = \theta_1 - \eta \frac{\partial \mathcal{L}}{\partial \theta_1}$$

$\vdots$

$$\theta_{N_S-1} = \theta_{N_S-2} - \eta \frac{\partial \mathcal{L}}{\partial \theta_{N_S-2}}$$

$$\theta_{N_S} = \theta_{N_S-1} - \eta \frac{\partial \mathcal{L}}{\partial \theta_{N_S-1}}$$

- Therefore, after N_S optimization steps:

$$\theta_{N_S} = \theta_{\text{past}} - \eta \sum_{i=0}^{N_S-1} \frac{\partial \mathcal{L}}{\partial \theta_i} \quad (2)$$

since $\theta_{\text{past}} = \theta_0$.

- Then, the θ_{past} is updated as follows:

$$\theta_{\text{past}} = (1 - \alpha)\theta_{\text{past}} + \alpha\theta_{N_S-1} = \theta_{\text{past}} - \alpha\eta \sum_{i=0}^{N_S-1} \frac{\partial \mathcal{L}}{\partial \theta_i} \quad (3)$$

- Therefore, after N_S optimization steps:

$$\theta_{N_S} = \theta_{\text{past}} - \eta \sum_{i=0}^{N_S-1} \frac{\partial \mathcal{L}}{\partial \theta_i} \quad (2)$$

since $\theta_{\text{past}} = \theta_0$.

- Then, the θ_{past} is updated as follows:

$$\theta_{\text{past}} = (1 - \alpha)\theta_{\text{past}} + \alpha\theta_{N_S-1} = \theta_{\text{past}} - \alpha\eta \sum_{i=0}^{N_S-1} \frac{\partial \mathcal{L}}{\partial \theta_i} \quad (3)$$

- So we simply reduce the learning rate by α ?

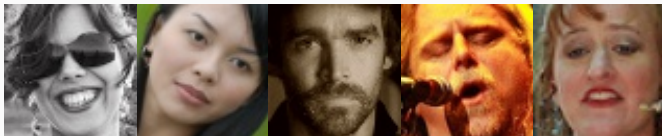
$$\theta_{\text{past}} = (1 - \alpha)\theta_{\text{past}} + \alpha\theta_{N_S-1} = \theta_{\text{past}} - \alpha\eta \sum_{i=0}^{N_S} \frac{\partial \mathcal{L}}{\partial \theta_i} \quad (4)$$

- We do not merely reduce learning rate by α !
 - The intermediate $N_S > 1$ states are calculated using the regular learning rate (solution space exploration)
 - When we update the stable state we lower the learning rate (reducing the risk of following a bad descent direction)!
- Even if we take **relatively large descent steps** (that overshoots the local minimum) the **stability will not be greatly affected!**

EXPERIMENTAL EVALUATION

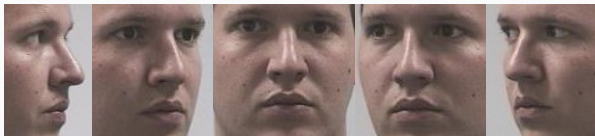
DATASETS

- Two datasets used for evaluating the proposed method
- **Annotated Facial Landmarks in the Wild dataset (AFLW)**
 - Continuous horizontal pose annotations



Head Pose Image Dataset (HPID)

- 13 discrete horizontal pose annotations



- During the training the following data augmentation techniques were used:
 1. random **vertical flip** with probability 0.5
 2. random **horizontal shift** up to 5%
 3. random **vertical shift** up to 5%
 4. random **zoom** up to 5%
 5. random **rotation** up to 10°

- For regressing the pose a **deep CNN** was used:

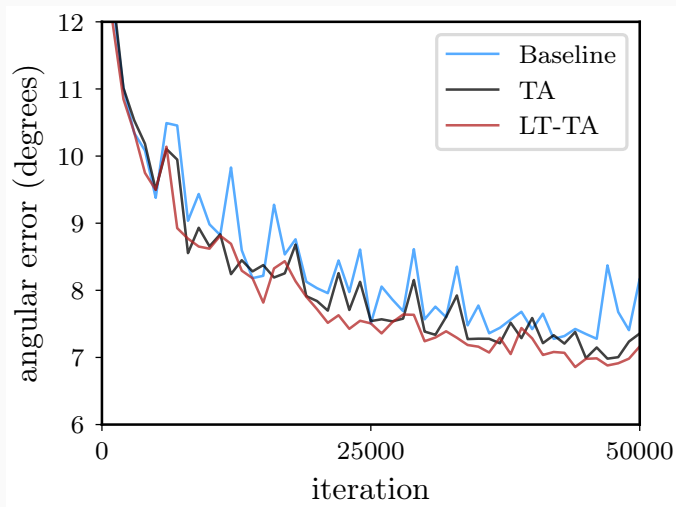
Layer Type	Output Shape
Input	$32 \times 32 \times 3$
Convolutional (3×3)	$30 \times 30 \times 16$
Convolutional (3×3)	$28 \times 28 \times 16$
Max Pooling (2×2)	$14 \times 14 \times 16$
LCN + Dropout ($p = 0.5$)	$14 \times 14 \times 16$
Convolutional (3×3)	$12 \times 12 \times 32$
Convolutional (3×3)	$10 \times 10 \times 32$
Max Pooling (2×2)	$5 \times 5 \times 32$
LCN + Dropout ($p = 0.5$)	$5 \times 5 \times 32$
Dense	256
Dense	1

- The images were resized to 32×32 before feeding them to network

- A **lightweight CNN architecture** was used (**less than 300K parameters**)
 - This allows the model to be deployed on **limited-resource scenarios**, such as embedded systems running on **drones** (e.g., NVIDIA TX-2)
- The networks were trained for 50,000 iterations using the Adam algorithm (the default hyper-parameters were used)
- Mini-batches of 32 samples were used

- The **proposed technique** (Long-Term Temporal Averaging, LT-TA) is compared to both **plain Temporal Averaging** (TA) and to **plain SGD** using Adam (Baseline)
- The mean angular error is reported
- The train error deviation (as a stability measure) is measured during the last 5,000 iterations

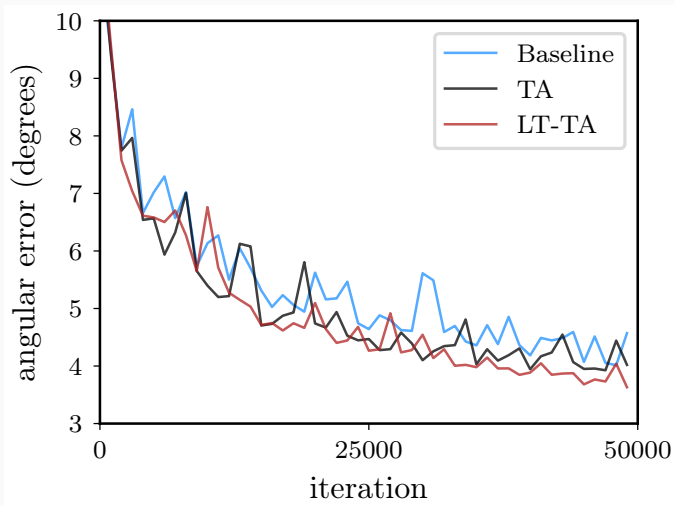
Method	Train Error	Train Deviation	Test Error
Baseline	7.41	0.32	8.16
TA	7.24	0.18	8.01
LT-TA	6.98	0.17	7.72



- Again, both the **training/test error** as well as **the stability of the optimization process** are improved when the proposed method is used

Method	Train Error	Train Deviation	Test Error
Baseline	4.57	0.21	5.99
TA	4.02	0.21	5.89
LT-TA	3.63	0.11	5.74

HPID EVALUATION



CONCLUSION

- A **Long-Term Temporal Averaging** was proposed and evaluated using two facial pose estimation datasets
 - Big descent steps are used to explore the solution space
 - An exponential running average is used to bias the current parameters towards stabler states
- The proposed method:
 - improves the **convergence speed**
 - improves **the stability** of the convergence process
 - reduces the risk of **stopping when a bad descent step was taken**
 - ensures that the network will **perform and generalize well** at any point of the training process

- The proposed method can be evaluated on other tasks and scenarios
 - Classification, regression, **reinforcement learning tasks**, etc.
 - Different architectures, networks, learning rates, etc.
- The **update rate** for the exponentially averaged parameters **can be dynamically determined** (e.g., using second-order statistics or using the relative change of the loss function)

This project has received funding from the European Union's Horizon 2020 research and innovation programme under grant agreement No 731667 (MULTIDRONE). This publication reflects the authors' views only. The European Commission is not responsible for any use that may be made of the information it contains.



THANK YOU!

QUESTIONS?